Generative AI and Applications

Course Code: 25CS60TP1378-3

L:3 Hrs, P: 2 Hr, Per Week Total Credits:4

Department of Computer Science and Applications

By Dr. Suhashini Chawrasia

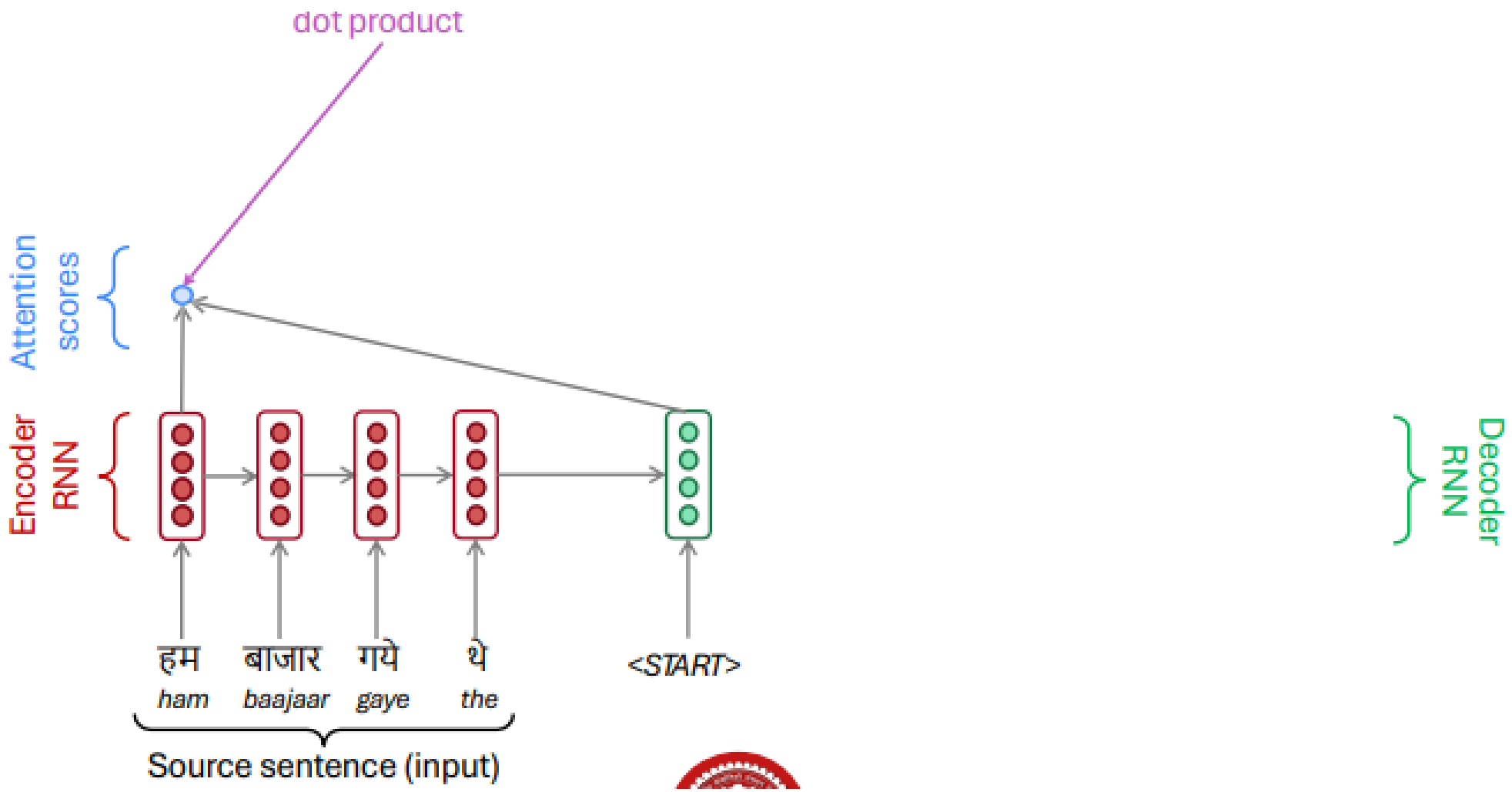
Unit-III

Attention Mechanism in Encoder-Decoder Architecture, Bahdanau Attention vs Luong Attention, Introduction to Transformers, Problems in encoder-decoder architecture with LSTM.

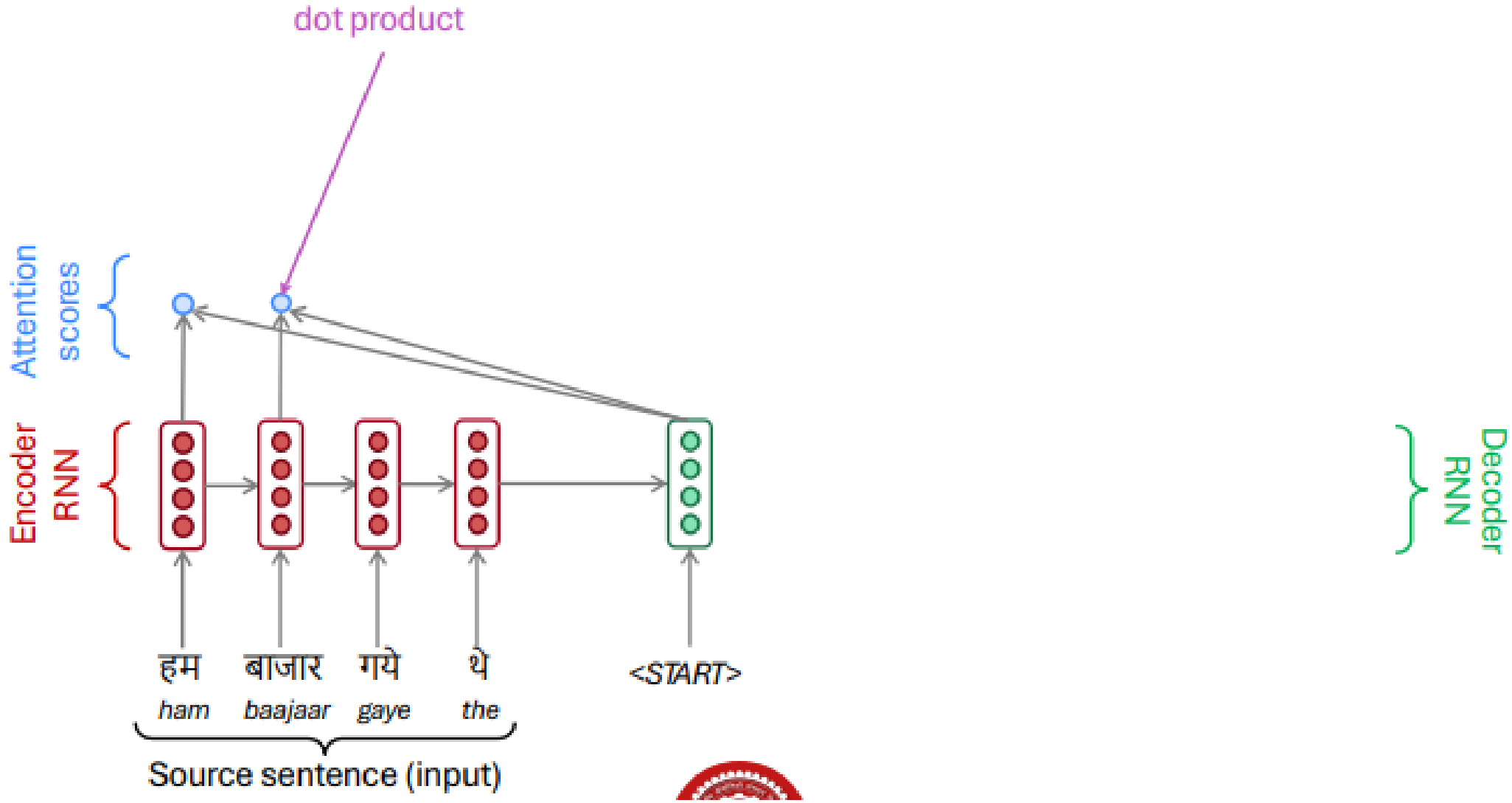
Attention

- Attention provides a solution to the bottleneck problem
- On each step of decoder use direct connections to the encoder to focus on a particular part of the source sequence
- Enhances the capability of Decoder-
  - Each state of the Decoder involves all the hidden states of Encoder.
  - Utilise relevant context from Encoder efficiently.
- Fact is, not all the parts of the input sequence are equally relevant to decoder steps.
- Attention mechanism addresses this-
  - Assign a probabilistic weight to each hidden state of the encoder.
  - Weight determines how much relevance (attention) each hidden state has.
  - Creating context-sensitive representation for each token during decoding.
- Attention mechanism functions like a memory. Stored info helps the decoder access relevant past inputs.
- Prevents information loss in seq-to-seq models.

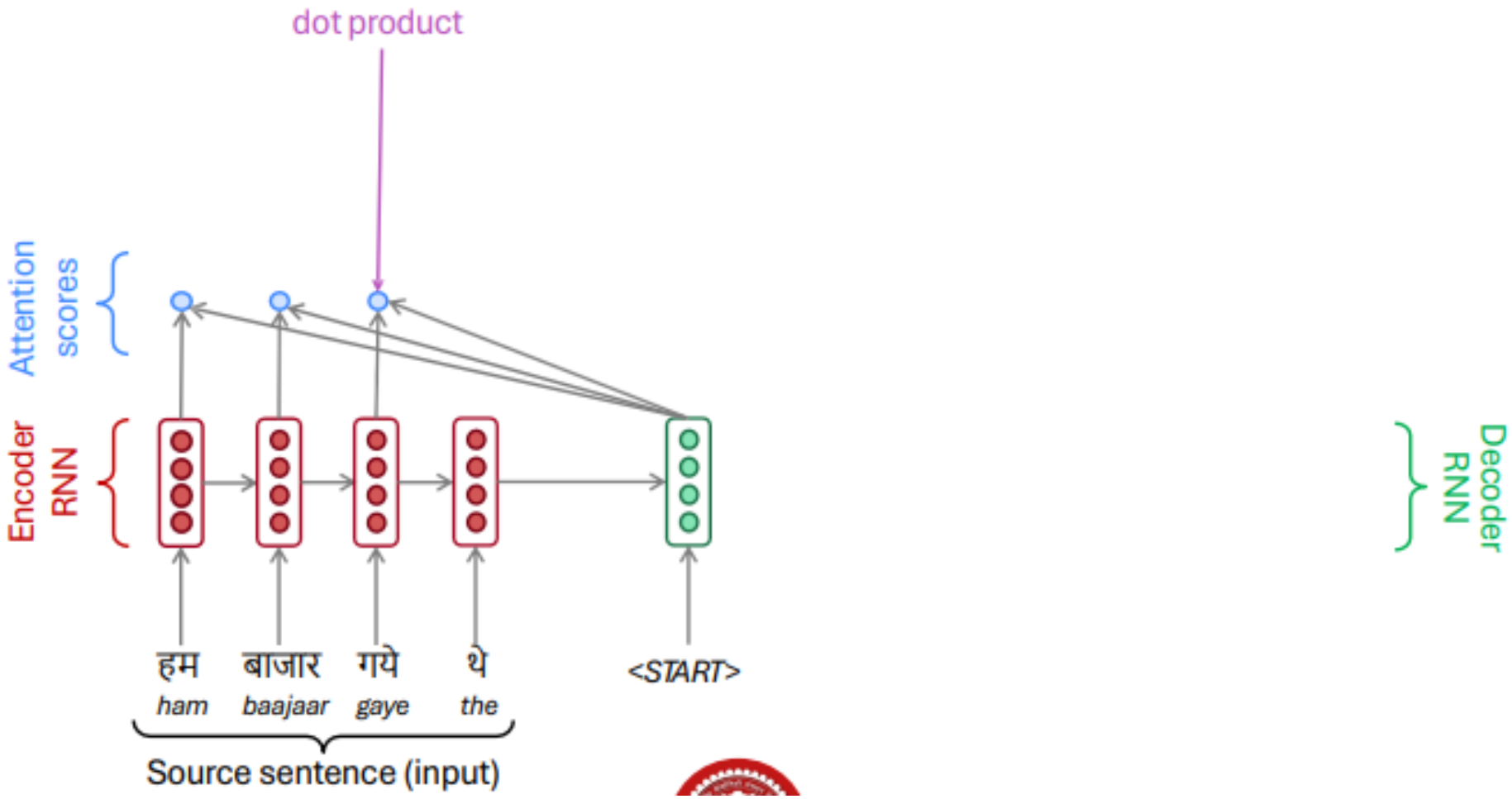
Architecture of Attention



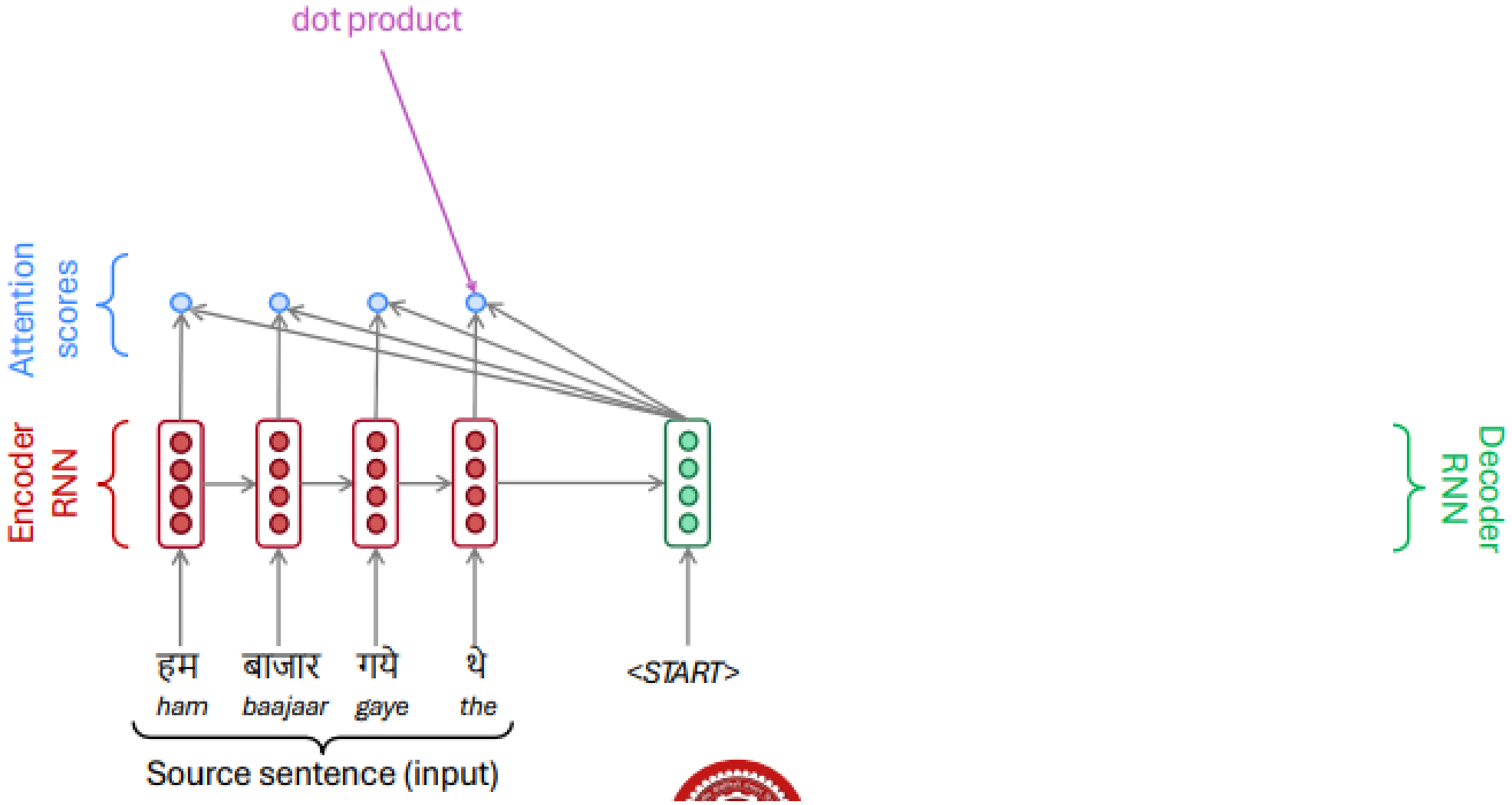
Architecture of Attention



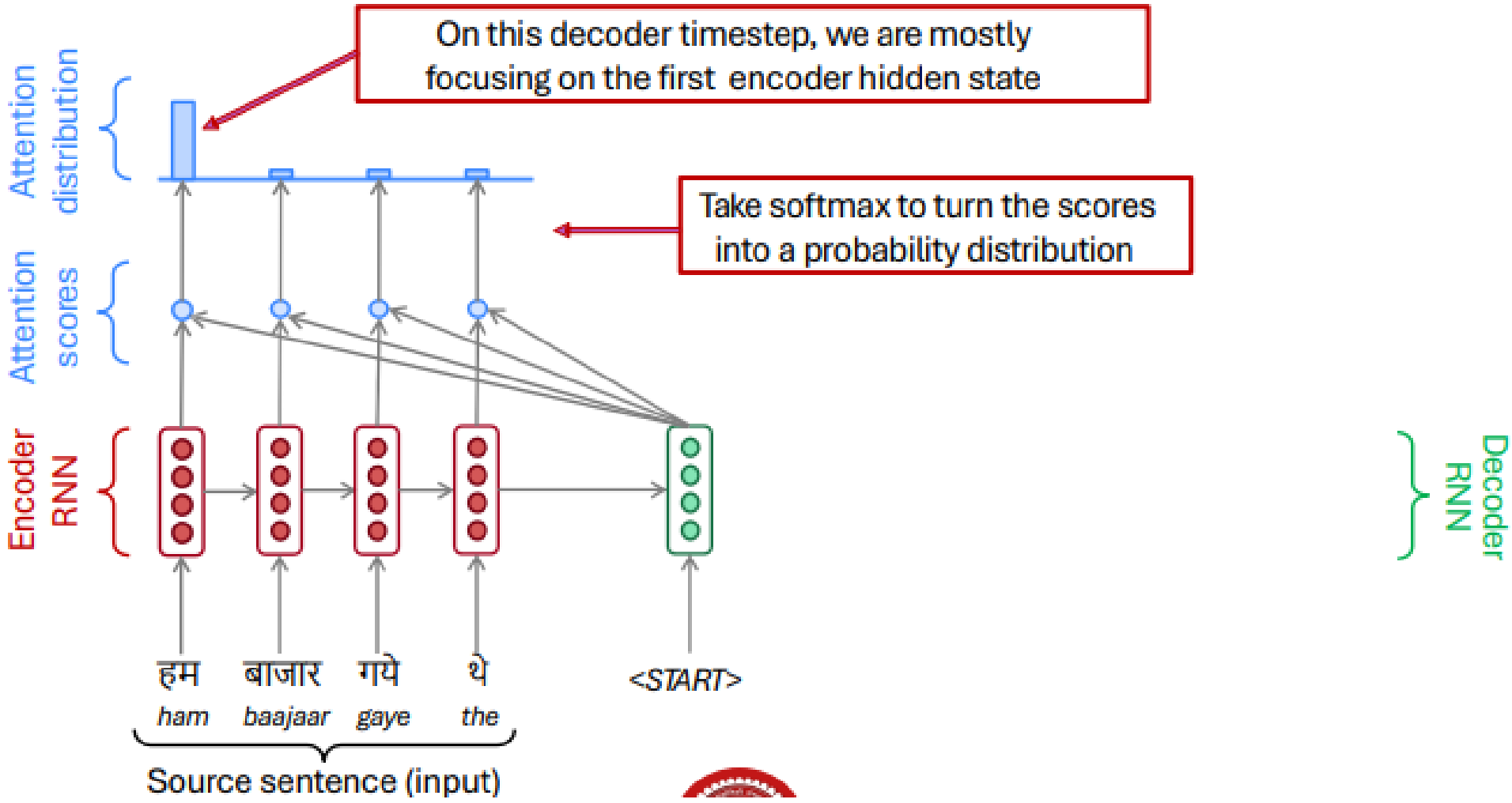
Architecture of Attention



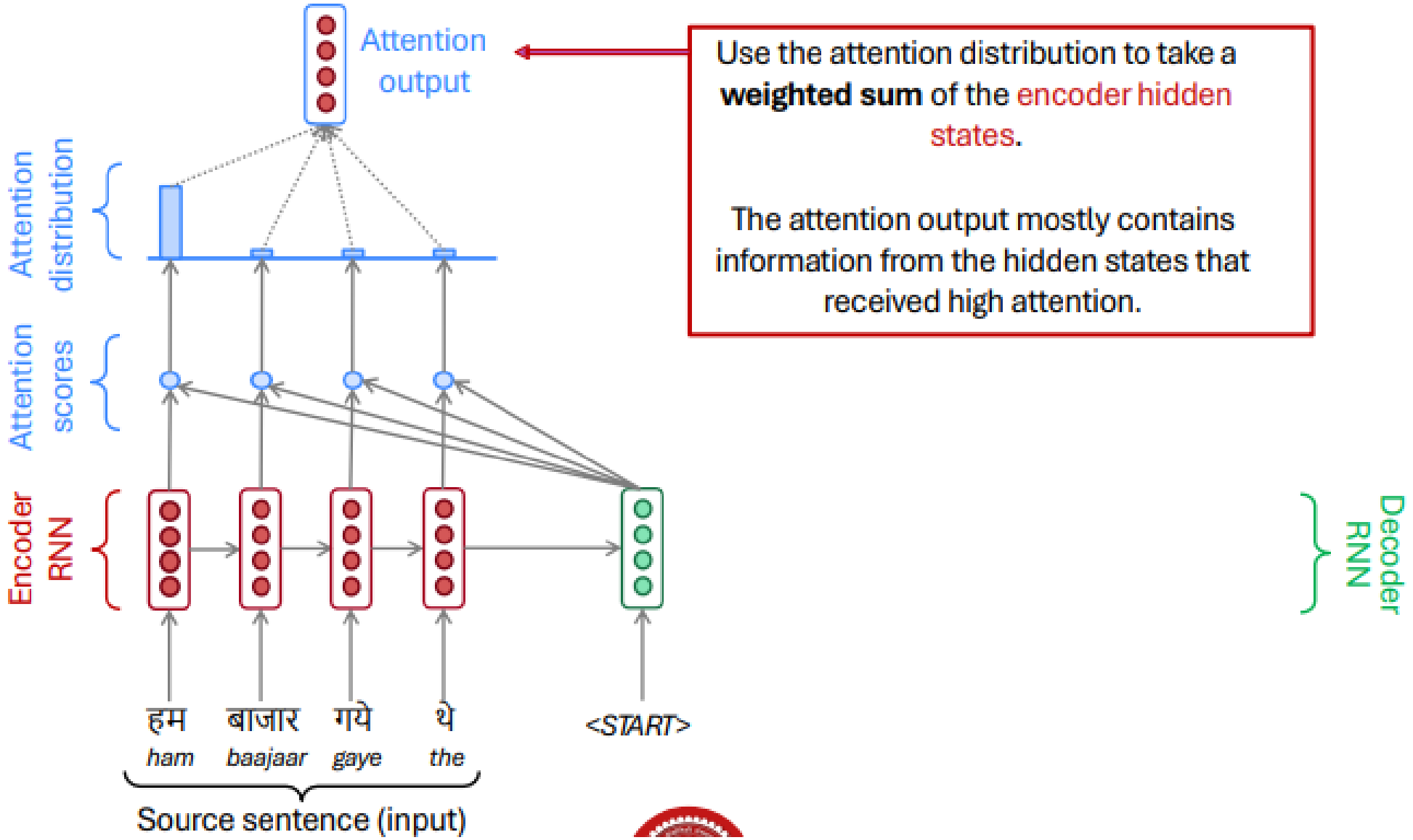
Architecture of Attention



Architecture of Attention



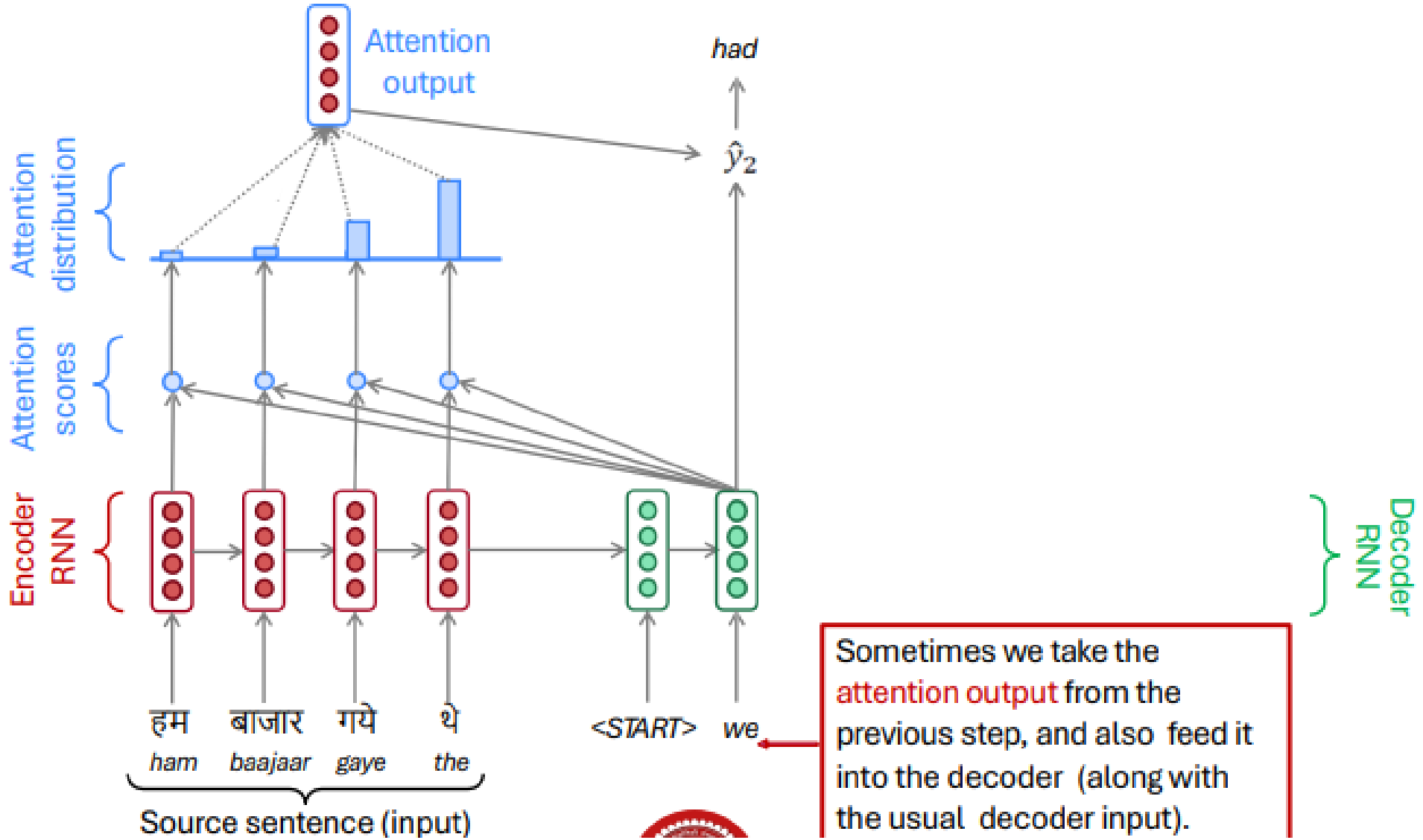
Architecture of Attention



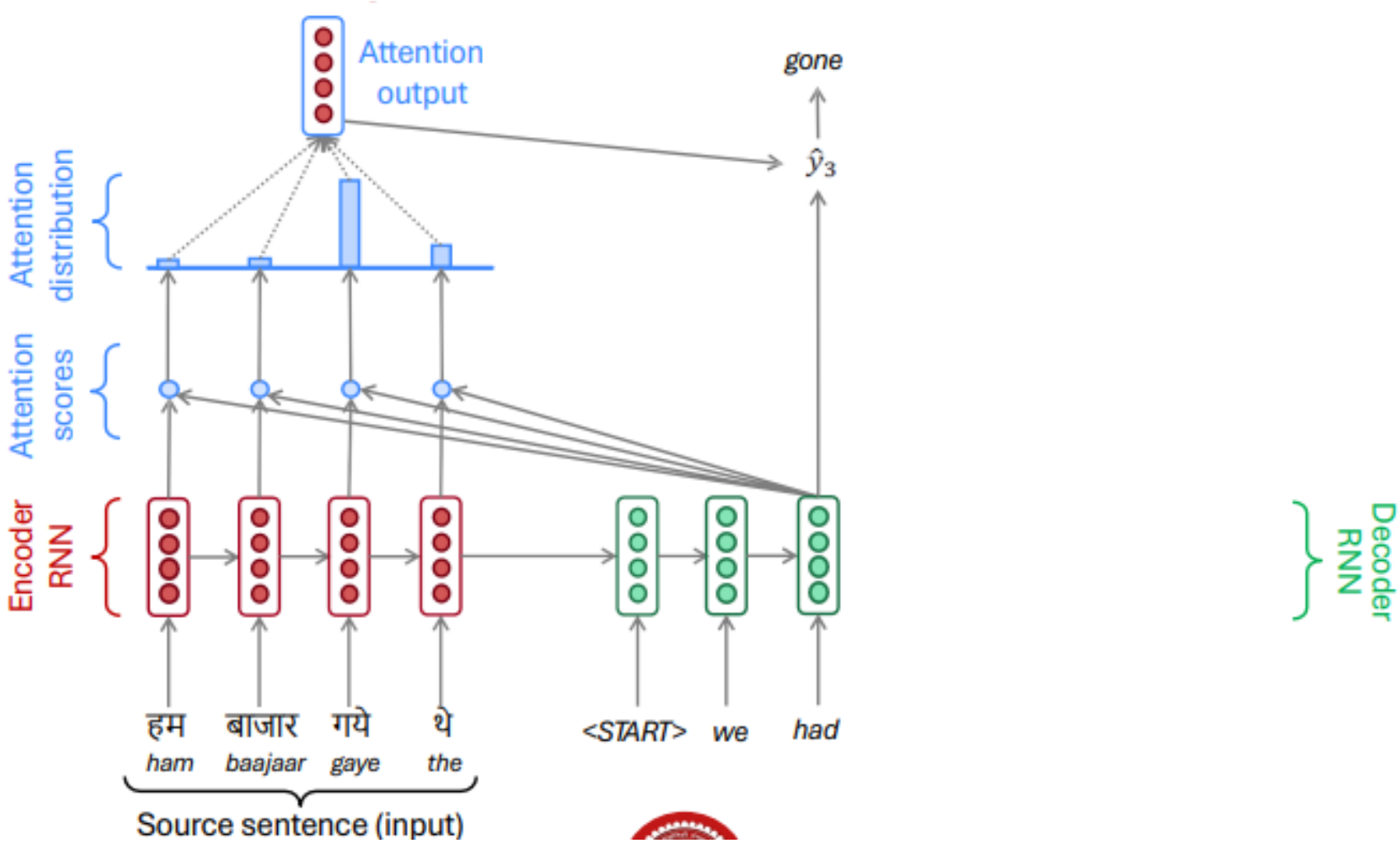
Use the attention distribution to take a **weighted sum** of the encoder hidden states.

The attention output mostly contains information from the hidden states that received high attention.

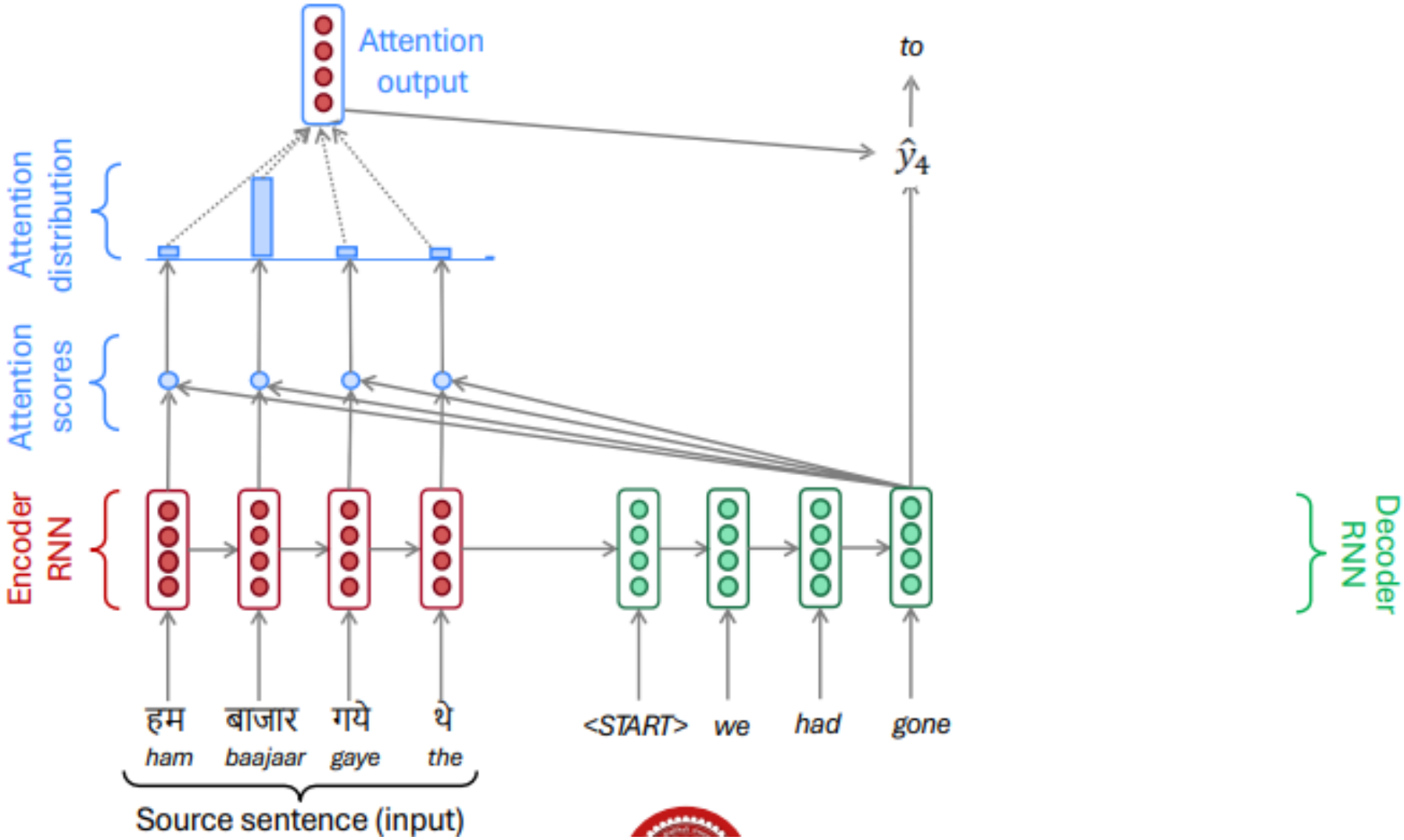
Architecture of Attention



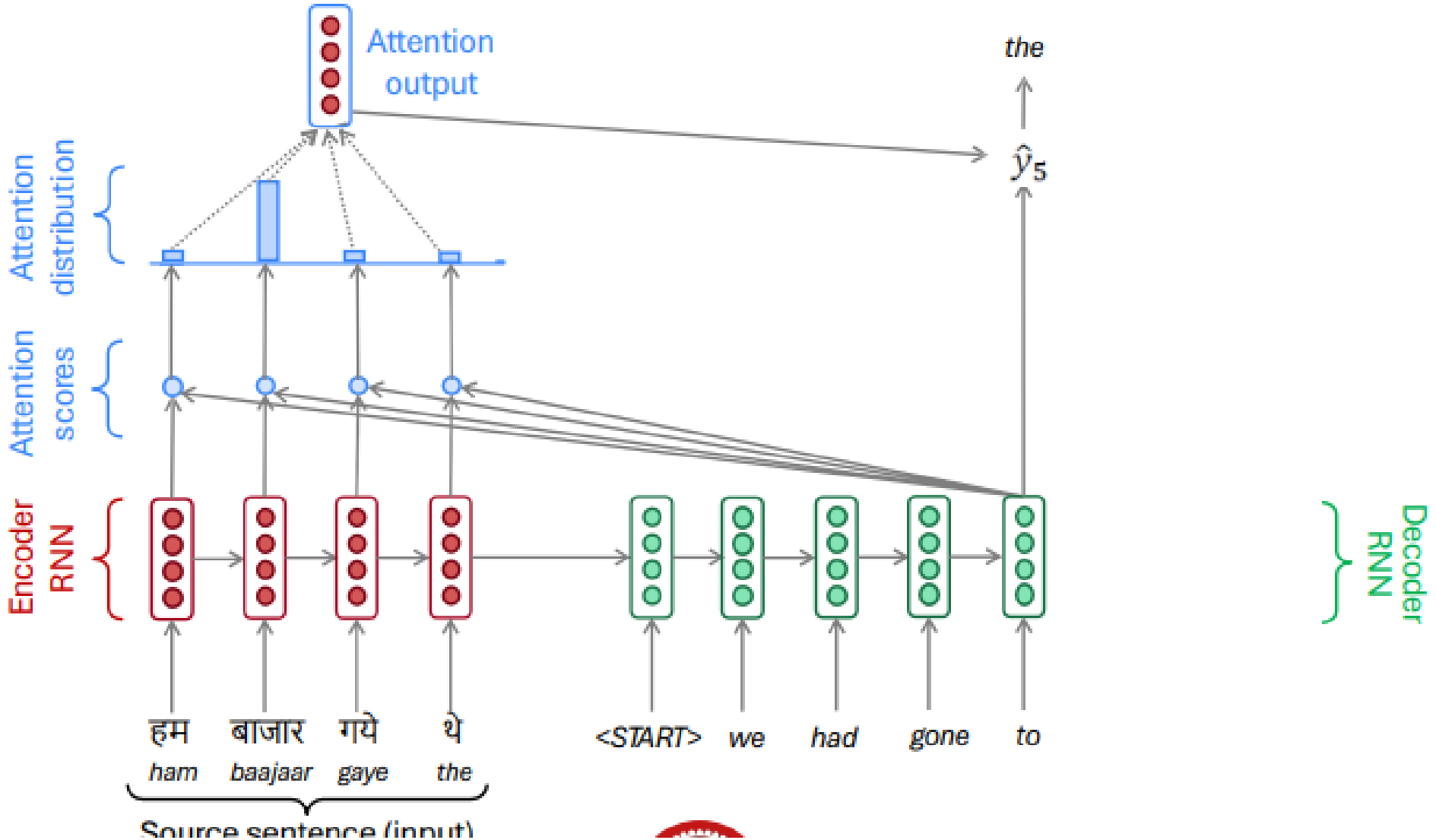
Architecture of Attention



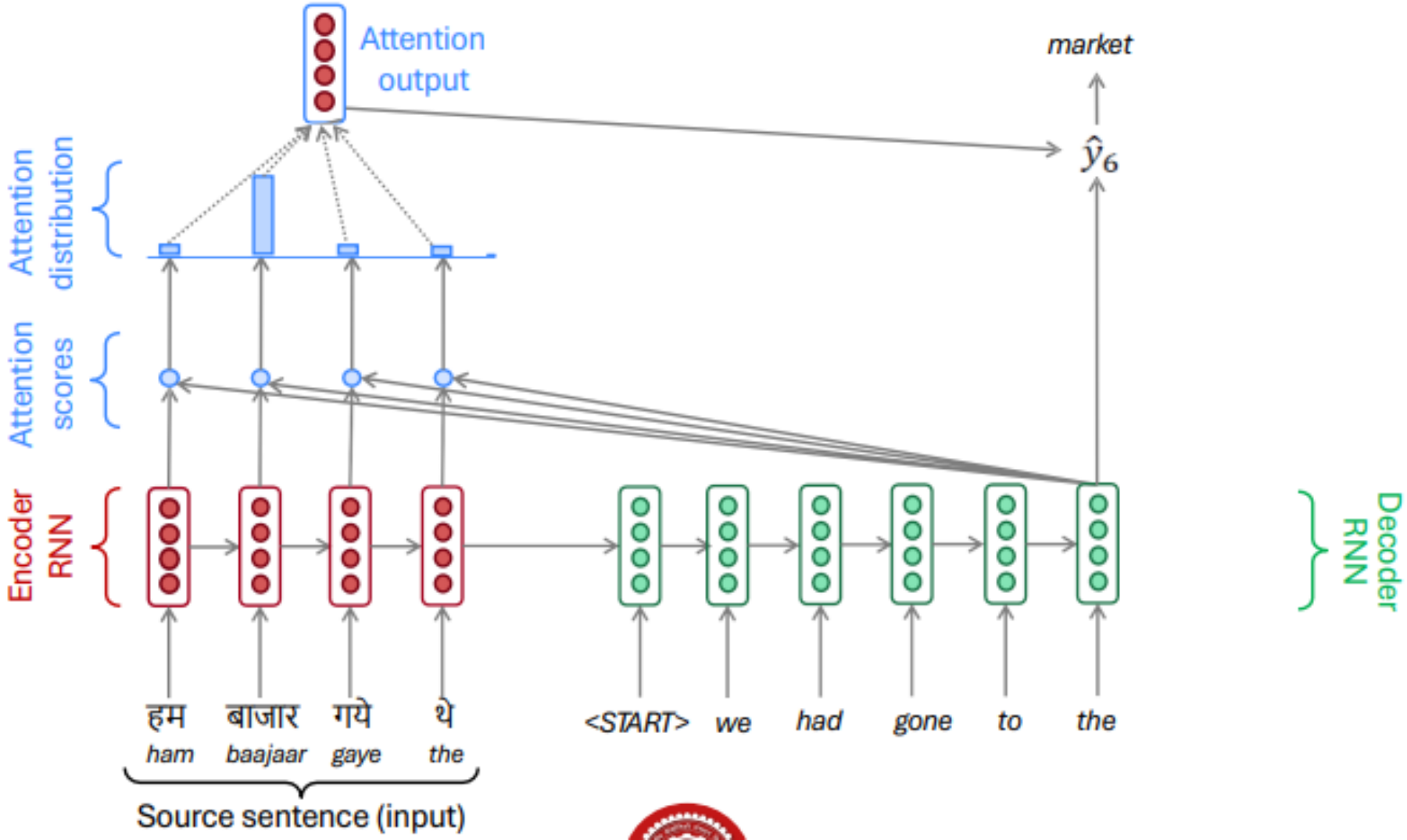
Architecture of Attention



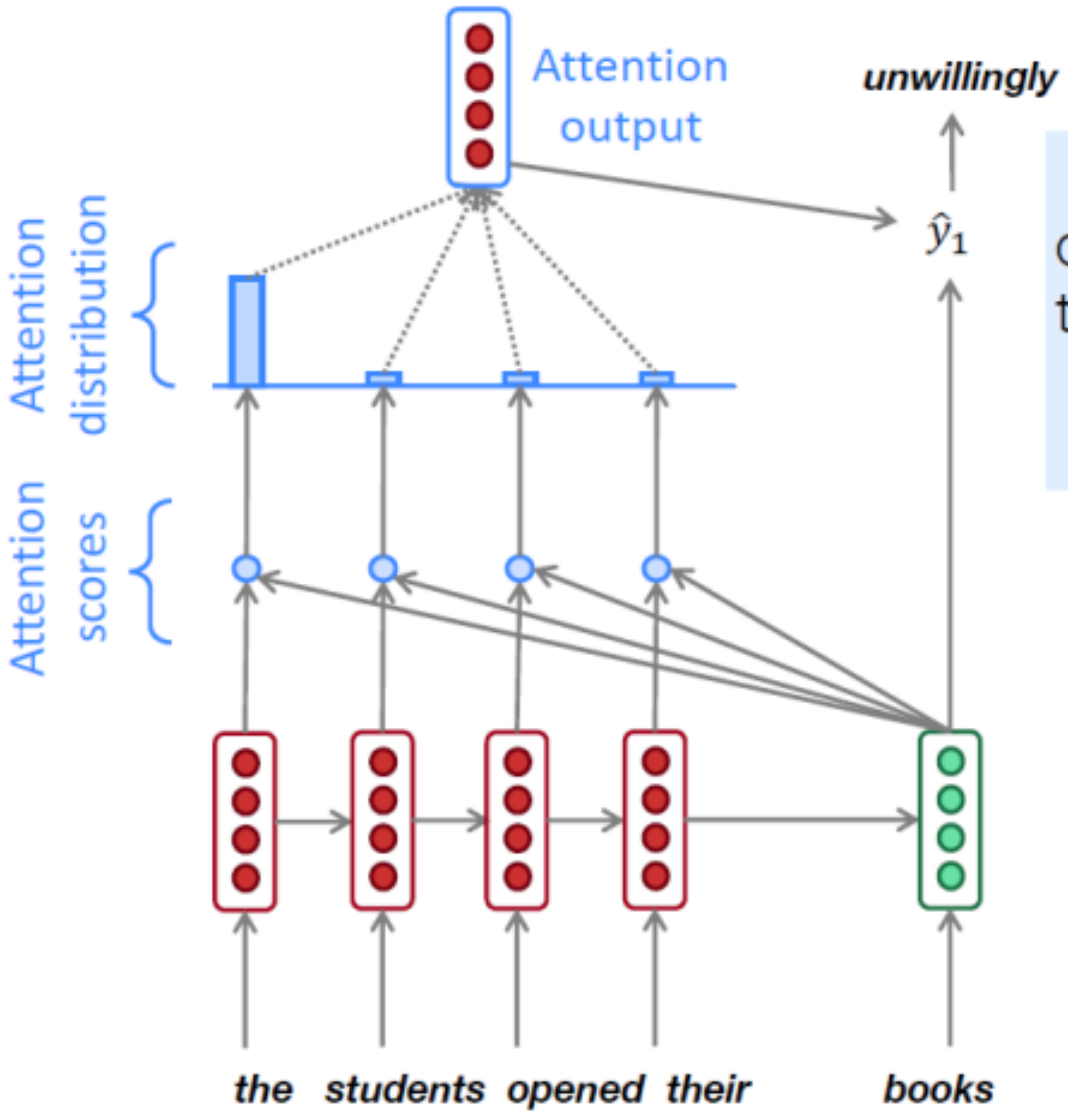
Architecture of Attention



Architecture of Attention



Architecture of Attention



Concatenate (or otherwise compose) the attention output with the current hidden state, then pass through a softmax layer to predict the next word

Mathematical formulation of Attention

- Encoder hidden states $h_1, \ldots, h_N \in R^h$
- On timestep t, decoder hidden state $s_t \in R^h$
- Attention scores e^t for this step:

  $e^t = [s_t^h h_{1,\ldots,} s_t^T h_N] \in R^h$
- Apply SoftMax to get the attention distribution α^t for this step (this is a probability distribution, sums to 1)

  $\alpha^t = \text{SoftMax}(e^t) = \frac{e^t}{\sum_{j=1}^{k} e^{t_j}} \in R^N$
- Use α^t to take a weighted sum of the encoder hidden states to get the attention output at a_t

  $a_t = \sum_{i=1}^{N} \alpha_i^t h_1 \in R^h$
- Finally concatenate the attention output a_t with the decoder hidden state s_t and proceed as in the non-attention seq2seq model

  $[a_t; s_t] \in R^{2h}$

First Problem based on attention

Assume that we are using an attention mechanism in an encoder-decoder architecture to translate the English input sequence “Night swallowed the stars” into the Italian output sequence “La notte inghiotti stelle”. Consider Encoder hidden states

$e^1 = [0.1, 0.2]$

$e^2 = [0.3, 0.8]$

$e^3 = [0.5, 0.6]$

$e^4 = [0.4, 0.2]$.

Give the step-by-step calculation of the attention output and hence the concatenation vector to predict the Italian word “notte”, having decoder hidden state $d^t = [0.5, 0.3]$

Solution: Attention solution word document in google class room

Assignment
Second Problem based on attention

Consider translating the English phrase ‘I love bananas’ into Spanish as ‘amo las bananas’. The hidden states of the encoders are [0.1,0.2], [0.4,0.8] and [0.5,0.6]. Compute attention score for decoder which outputs the Spanish word ‘las’ having a hidden state of [0.4,0.6]

Answer: [0.32, 0.57, 0.4, 0.6]

Importance of the attention mechanism in RNN-based Encoder-Decoder models for sequence-to-sequence

Importance of Attention

- Attention significantly improves NMT performance
- It’s very useful to allow decoder to focus on certain parts of the source
- Attention solves the bottleneck problem
- Attention allows decoder to look directly at source; bypass bottleneck
- Attention helps with vanishing gradient problem
- Attention provides shortcut to faraway states
- Attention provides some interpretability
- By inspecting attention distribution, decoder focus on
- Attention get (soft) alignment for free
- Attention never explicitly trained an alignment system
- In Attention, the network just learned alignment by itself

Importance of Attention

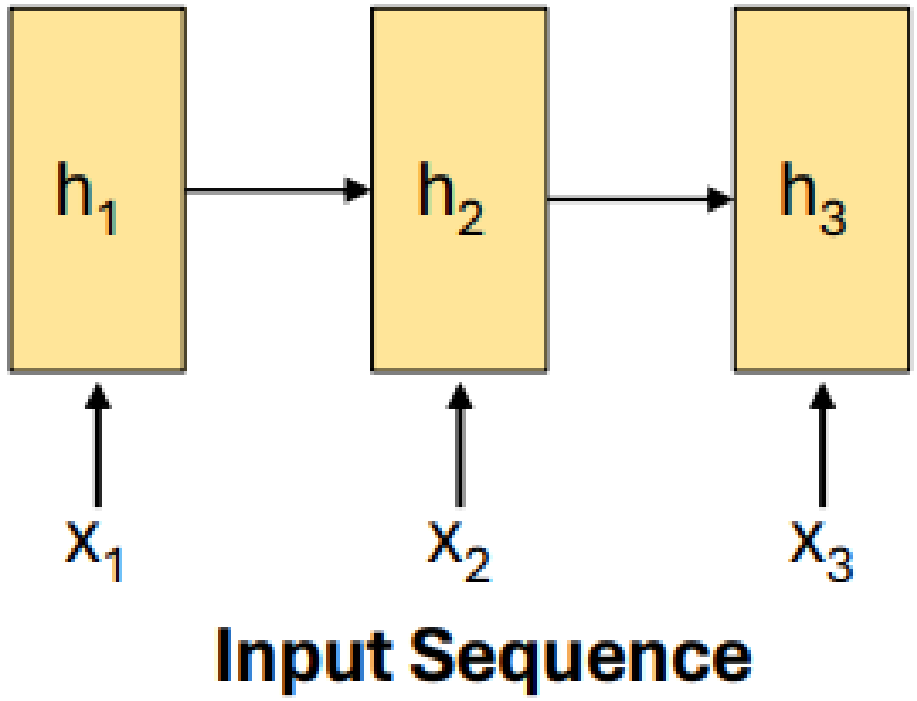
- Attention is a General Deep Learning Technique
- Attention is a great way to improve the sequence-to-sequence model for Machine Translation
- Attention can be used in many architectures (not just seq2seq) and many tasks (not just MT)
- Given a set of vector values, and a vector query, attention is a technique to compute a weighted sum of the
- values, dependent on the query
- Query attends to the values
- For example, in the seq2seq + attention model, each decoder hidden state (query) attends to all the encoder hidden states (values)
- Intuition:
- The weighted sum is a selective summary of the information contained in the values, where the query determines which values to focus on
- Attention is a way to obtain a fixed-size representation of an arbitrary set of representations (the values), dependent on some other representation (the query)

Limitations of Neural Language Models

- Require vast amount of data.
- Need high computational power and memory.
- Biasing can be present in the output (as it learns and propagate the biases further).
  - Ex. Gender bias – “Women are nurses”. NLM may not recognise that men can also be nurses. (similarly age, nationality, profession, etc…)
- Another issues:
  - Hallucination – non-factual contents in predicted output.
  - Overfitting – due to overtraining.

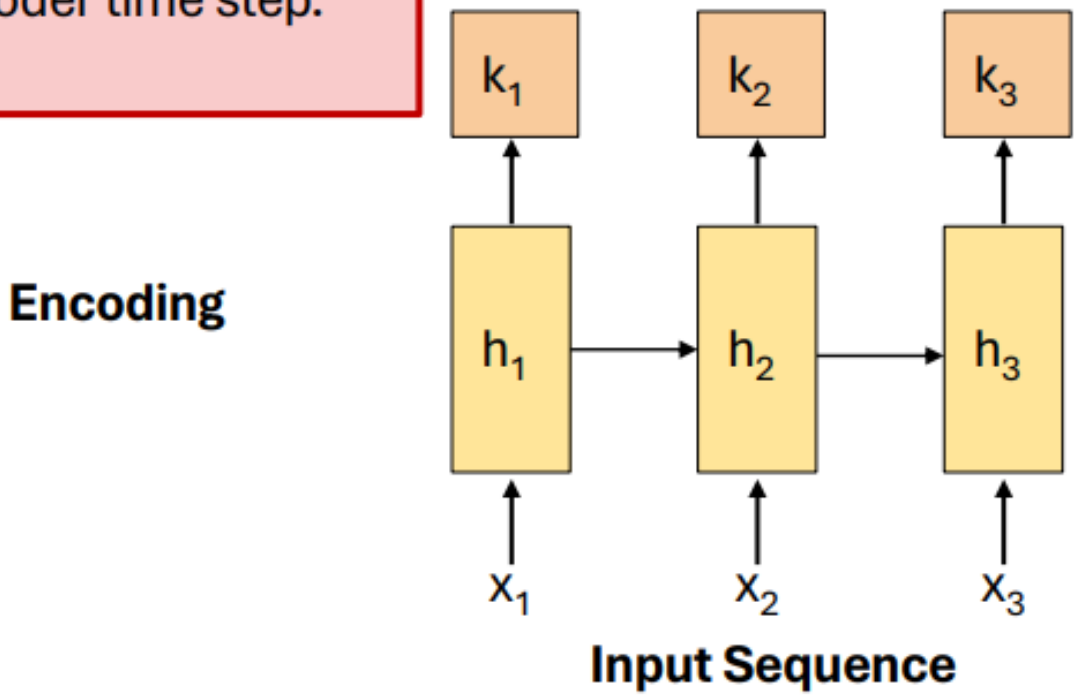
Attention

Encoding

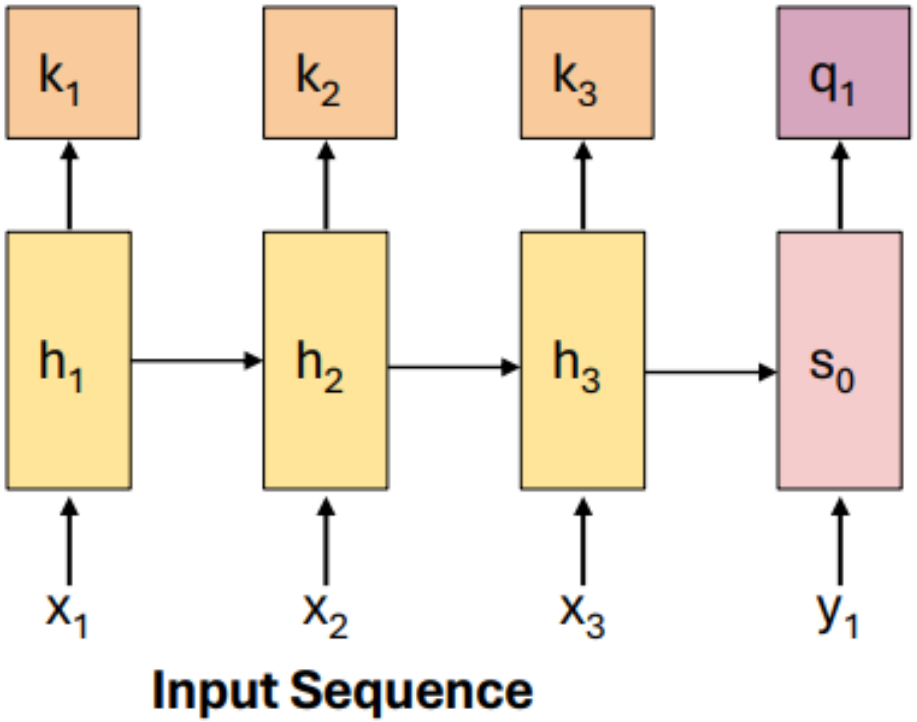


Attention

Key vectors represent what **information** is **encoded** at each encoder time step.

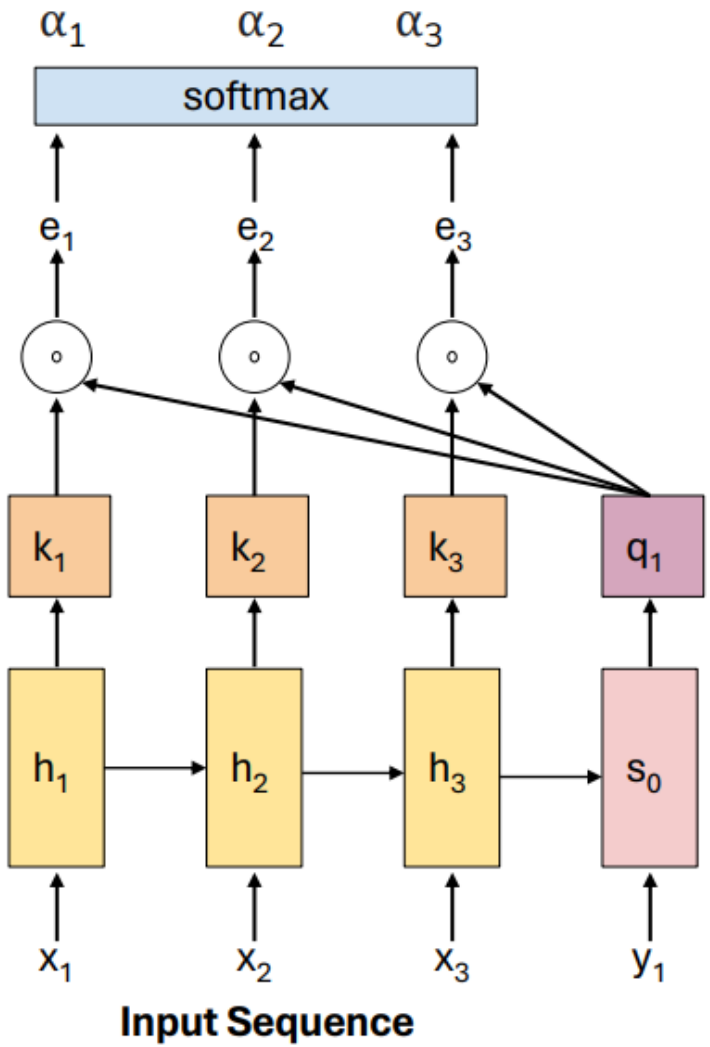


Attention



Query vectors represent what information we are **looking for** at each decoder time step.

Attention

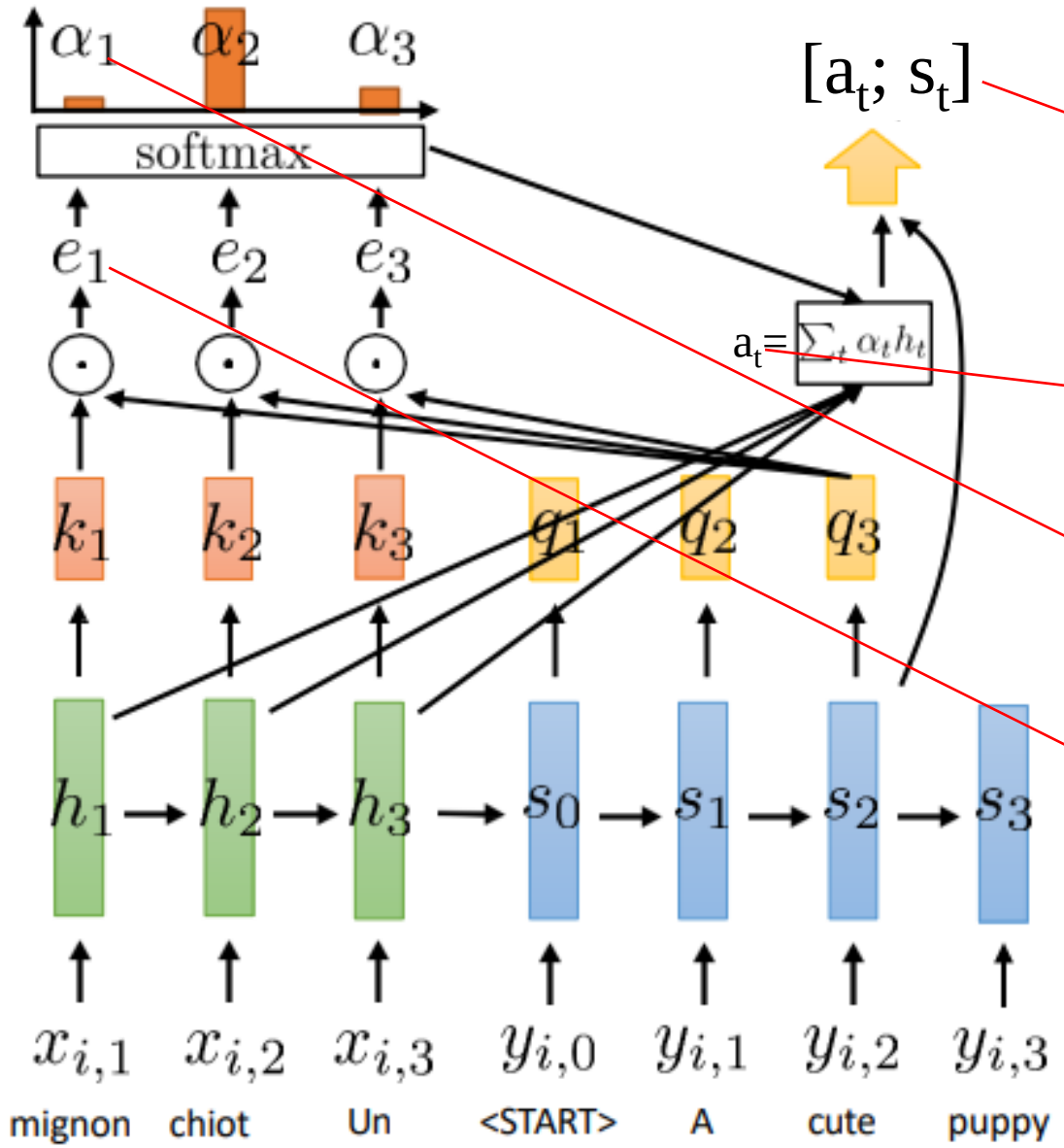


Softmax converts the similarity scores into a **probability distribution**.

Dot product between query vector and every key vector gives **similarity score**.

Decoding

Attention



- Finally concatenate the attention output a_t with the decoder hidden state s_t and proceed

  as in the non-attention seq2seq model

  $[a_t;s_t] \in R^{2h}$
- Use α^t to take a weighted sum of the encoder hidden states to get the attention output at a_t

  $a_t = \sum_t \alpha_t h_t \in Rh$
- Apply softmax to get the attention distribution α_t for this step (this is a probability distribution, sums to 1)

  $\alpha_t = \text{softmax}(e_t) = \frac{e_t}{\sum_{j=1}^{k} e_{t_j}} \in R^h$
- Attention scores e^t for this step:

  $e_t = [s_t h_{1,\ldots,} s_t h_N] \in R^h$
- On timestep t, decoder hidden state $s_t \in R^h$
- Encoder hidden states $h_1, \ldots, h_N \in R^h$

Variants of Attention

- Attention
- Self Attention
- Multi head Attention
- Cross Attention

Introduction to Transformer

Transformer models operate on the principle of next-token prediction: given a text prompt from the user, the most probable next token that will follow the input will be generated. The core innovation and power of Transformers lie in their use of self-attention mechanism, which allows them to process entire sequences and capture long-range dependencies more effectively than previous architectures.

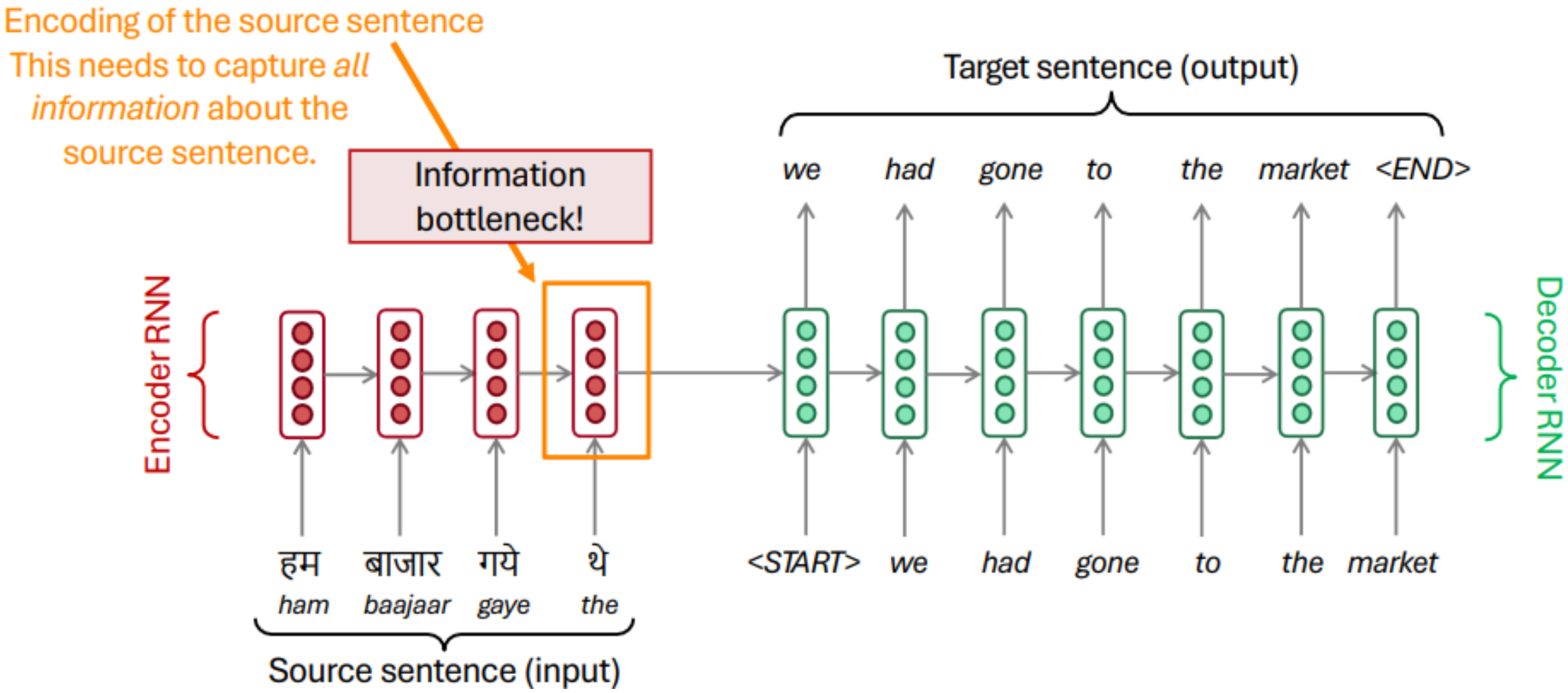
Problems with encoder decoder architecture with LSTM

- Lack of parallelizability
- Hard to scale
- Bottleneck problem
- Poor Long-Range Dependencies
- Exposure Bias during Inference
- Vanishing and Exploding Gradients over Deep Layers
- Tremendous processing time for huge data set.

Bottleneck problem

- **Fixed-Length Constraint**: The encoder compresses the entire input sequence—no matter how long—into a single final hidden layer
- **Loss of Detail**: Forcing a highly detailed, 50-word sentence or a long paragraph into a small, rigid vector means a massive amount of nuance and structural detail gets squeezed out, drastically hurting the decoder's accuracy

Bottleneck problem



Poor Long-Range Dependencies

- **Forgetting Early Tokens**: LSTMs utilize gating mechanisms to mitigate the vanishing gradient problem, but they still suffer from a practical memory limit
- **Recency Bias**: Because processing happens sequentially, the final context vector is naturally biased toward the tokens at the very end of the input sequence. The network frequently "forgets" foundational context from the beginning of a long text.

Lack of parallelizability

- **Sequential Processing**: LSTMs must process data step-by-step. To compute the hidden state at time step t, the network absolutely requires the hidden state from time step t-1.
- **Hardware Inefficiency**: This sequential nature prevents training from being parallelized across modern GPUs. Consequently, training these models on massive datasets is incredibly slow and resource-intensive compared to modern architectures

Exposure Bias during Inference (testing)

- **Teacher Forcing Discrepancy**: During training, a technique called Teacher Forcing is usually used, where the decoder is fed the ground-truth target word from the previous step.
- **Compounding Errors**: During inference (real-world testing), the ground truth is unavailable, so the decoder must rely on its own predicted token from the previous step. If the decoder makes a single mistake early on, that error feeds into the next step, causing a cascade of compounding errors throughout the rest of the generation.

Vanishing and Exploding Gradients over Deep Layers

- **Deep Architecture Limits**: While an individual LSTM cell handles gradients better than a basic RNN, stacking multiple LSTM layers to create a "Deep" Encoder-Decoder network reintroduces vanishing or exploding gradients.
- **Diminishing Returns**: Backpropagating errors through both time steps and multiple structural layers makes the network notoriously unstable and difficult to train.

Introduction to Transformer

- To address the above issues, we need to remove dependencies on the output of the previous timestep.
- Solution: Vaswani et al. (2017) in “Attention Is All you Need” introduced the concept of “self-attention”.
- This architecture is named as “Transformer”.
- GPT-2 family of models are prominent examples of text-generative Transformers. Transformer Explainer is powered by the GPT-2 (small) model which has 124 million parameters. While it is not the latest or most powerful Transformer model, it shares many of the same architectural components and principles found in the current state-of-the-art models making it an ideal starting point for understanding the basics.

Three key components of Transformer

- **Embedding**: Text input is divided into smaller units called tokens, which can be words or subwords. These tokens are converted into numerical vectors called embeddings, which capture the semantic meaning of words.
- **Transformer Block** is the fundamental building block of the model that processes and transforms the input data. Each block includes:
  - Attention Mechanism, the core component of the Transformer block. It allows tokens to communicate with other tokens, capturing contextual information and relationships between words.
  - MLP (Multilayer Perceptron) Layer, a feed-forward network that operates on each token independently. While the goal of the attention layer is to route information between tokens, the goal of the MLP is to refine each token's representation.
- **Output Probabilities**: The final linear and softmax layers transform the processed embeddings into probabilities, enabling the model to make predictions about the next token in a sequence.

First component of Transformer: Embedding

To generate text using a Transformer model the input needs to be converted into a format that the model can understand and process. That is where embedding comes in: it transforms the text into a numerical representation that the model can work with. Following are the steps to convert a prompt into embedding are

1. Tokenization
2. Token embeddings
3. Positional encoding
4. Final embedding

Step 1: Tokenization

Tokenization is the process of breaking down the input text into smaller, more manageable pieces called tokens. These tokens can be a word or a subword. The words "Data" and "visualization" correspond to unique tokens, while the word "empowers" is split into two tokens. The full vocabulary of tokens is decided before training the model: GPT-2's vocabulary has 50,257 unique tokens. Split the input text into tokens with distinct IDs, called vector representation from embeddings.

Step 2: Token Embedding

GPT-2 (small) represents each token in the vocabulary as a 768-dimensional vector; the dimension of the vector depends on the model. These embedding vectors are stored in a matrix of shape (50,257, 768), containing approximately 39 million parameters. This extensive matrix allows the model to assign semantic meaning to each token, in the sense that tokens with similar usage or meaning in language are placed close together in this high-dimensional space, while dissimilar tokens are farther apart.

Step 3: Positional Encoding

The Embedding layer also encodes information about each token's position in the input prompt. Different models use various methods for positional encoding. GPT-2 trains its own positional encoding matrix from scratch, integrating it directly into the training process.

Step 4: Final Embedding

Finally, the tokens and positional encodings generates the final embedding representation. This combined representation captures both the semantic meaning of the tokens and their position in the input sequence.

Second component of transformer: Transformer Block

The core of the Transformer's processing lies in the Transformer block, which comprises multi-head self-attention and a Multi-Layer Perceptron layer. Most models consist of multiple such blocks that are stacked sequentially one after the other. The token representations evolve through layers, from the first block to the last one, allowing the model to build up an intricate understanding of each token. This layered approach leads to higher-order representations of the input. The GPT-2 (small) model consists of 12 such blocks.

Third component of transformer: Output Probabilities

After the input has been processed through all Transformer blocks, the output is passed through the final linear layer to prepare it for token prediction. This layer projects the final representations into a 50,257 dimensional space, where every token in the vocabulary has a corresponding value called logit. Any token can be the next word, so this process allows the model to simply rank these tokens by their likelihood of being that next word. Then apply the softmax function to convert the logits into a probability distribution that sums to one. This will allow the sample next token based on its likelihood.

Transformer Architecture

"Attention Is All You Need" paper in 2017 by google Mind

